8.5 Case Study: An Expression Tree

In Example 8.7, we introduced the use of a binary tree to represent the structure of an arithmetic expression. In this section, we define a new ExpressionTree class that provides support for constructing such trees, and for displaying and evaluating the arithmetic expression that such a tree represents. Our ExpressionTree class is defined as a subclass of LinkedBinaryTree, and we rely on the nonpublic mutators to construct such trees. Each internal node must store a string that defines a binary operator (e.g., `'+'`), and each leaf must store a numeric value (or a string representing a numeric value).

Our eventual goal is to build arbitrarily complex expression trees for compound arithmetic expressions such as $(((3+1)\times 4)/((9-5)+2))$. However, it suffices for the ExpressionTree class to support two basic forms of initialization:

ExpressionTree(value): Create a tree storing the given value at the root.

ExpressionTree(op, E_1, E_2): Create a tree storing string op at the root (e.g., $+$), and with the structures of existing ExpressionTree instances E_1 and E_2 as the left and right subtrees of the root, respectively.

Such a constructor for the ExpressionTree class is given in Code Fragment 8.35. The class formally inherits from LinkedBinaryTree, so it has access to all the nonpublic update methods that were defined in Section 8.3.1. We use `_add_root` to create an initial root of the tree storing the token provided as the first parameter. Then we perform run-time checking of the parameters to determine whether the caller invoked the one-parameter version of the constructor (in which case, we are done), or the three-parameter form. In that case, we use the inherited `_attach` method to incorporate the structure of the existing trees as subtrees of the root.

Composing a Parenthesized String Representation

A string representation of an existing expression tree instance, for example, as `'(((3+1)x4)/((9-5)+2))'`, can be produced by displaying tree elements using an inorder traversal, but with opening and closing parentheses inserted with a preorder and postorder step, respectively. In the context of an ExpressionTree class, we support a special `__str__` method (see Section 2.3.2) that returns the appropriate string. Because it is more efficient to first build a sequence of individual strings to be joined together (see discussion of "Composing Strings" in Section 5.4.2), the implementation of `__str__` relies on a nonpublic, recursive method named `_parenthesize_recur` that appends a series of strings to a list. These methods are included in Code 8.35.

```
class ExpressionTree(LinkedBinaryTree):
  """An arithmetic expression tree."""

  def __init__(self, token, left=None, right=None):
    """Create an expression tree.

    In a single parameter form, token should be a leaf value (e.g., '42'),
    and the expression tree will have that value at an isolated node.

    In a three-parameter version, token should be an operator,
    and left and right should be existing ExpressionTree instances
    that become the operands for the binary operator.
    """
    super().__init__( )                          # LinkedBinaryTree initialization
    if not isinstance(token, str):
      raise TypeError('Token must be a string')
    self._add_root(token)                        # use inherited, nonpublic method
    if left is not None:                         # presumably three-parameter form
      if token not in '+-*x/':
        raise ValueError('token must be valid operator')
      self._attach(self.root( ), left, right)    # use inherited, nonpublic method

  def __str__(self):
    """Return string representation of the expression."""
    pieces = [ ]                                 # sequence of piecewise strings to compose
    self._parenthesize_recur(self.root( ), pieces)
    return ''.join(pieces)

  def _parenthesize_recur(self, p, result):
    """Append piecewise representation of p's subtree to resulting list."""
    if self.is_leaf(p):
      result.append(str(p.element( )))                    # leaf value as a string
    else:
      result.append('(')                                  # opening parenthesis
      self._parenthesize_recur(self.left(p), result)      # left subtree
      result.append(p.element( ))                         # operator
      self._parenthesize_recur(self.right(p), result)     # right subtree
      result.append(')')                                  # closing parenthesis
```

Code Fragment 8.35: The beginning of an ExpressionTree class.

Expression Tree Evaluation

The numeric evaluation of an expression tree can be accomplished with a simple application of a postorder traversal. If we know the values represented by the two subtrees of an internal position, we can calculate the result of the computation that position designates. Pseudo-code for the recursive evaluation of the value represented by a subtree rooted at position p is given in Code Fragment 8.36.

```
Algorithm evaluate_recur(p):
  if p is a leaf then
    return the value stored at p
  else
    let ∘ be the operator stored at p
    x = evaluate_recur(left(p))
    y = evaluate_recur(right(p))
    return x ∘ y
```

Code Fragment 8.36: Algorithm evaluate_recur for evaluating the expression represented by a subtree of an arithmetic expression tree rooted at position p.

To implement this algorithm in the context of a Python ExpressionTree class, we provide a public evaluate method that is invoked on instance T as T.evaluate( ). Code Fragment 8.37 provides such an implementation, relying on a nonpublic _evaluate_recur method that computes the value of a designated subtree.

```
39  def evaluate(self):
40    """Return the numeric result of the expression."""
41    return self._evaluate_recur(self.root( ))
42
43  def _evaluate_recur(self, p):
44    """Return the numeric result of subtree rooted at p."""
45    if self.is_leaf(p):
46      return float(p.element( ))          # we assume element is numeric
47    else:
48      op = p.element( )
49      left_val = self._evaluate_recur(self.left(p))
50      right_val = self._evaluate_recur(self.right(p))
51      if op == '+': return left_val + right_val
52      elif op == '-': return left_val - right_val
53      elif op == '/': return left_val / right_val
54      else: return left_val * right_val       # treat 'x' or '*' as multiplication
```

Code Fragment 8.37: Support for evaluating an ExpressionTree instance.

Building an Expression Tree

The constructor for the ExpressionTree class, from Code Fragment 8.35, provides basic functionality for combining existing trees to build larger expression trees. However, the question still remains how to construct a tree that represents an expression for a given string, such as `'(((3+1)x4)/((9-5)+2))'`.

To automate this process, we rely on a bottom-up construction algorithm, assuming that a string can first be tokenized so that multidigit numbers are treated atomically (see Exercise R-8.30), and that the expression is fully parenthesized. The algorithm uses a stack S while scanning tokens of the input expression E to find values, operators, and right parentheses. (Left parentheses are ignored.)

- When we see an operator $\circ$, we push that string on the stack.
- When we see a literal value v, we create a single-node expression tree T storing v, and push T on the stack.
- When we see a right parenthesis, `')'`, we pop the top three items from the stack S, which represent a subexpression $(E_1 \circ E_2)$. We then construct a tree T using trees for E_1 and E_2 as subtrees of the root storing $\circ$, and push the resulting tree T back on the stack.

We repeat this until the expression E has been processed, at which time the top element on the stack is the expression tree for E. The total running time is $O(n)$.

An implementation of this algorithm is given in Code Fragment 8.38 in the form of a stand-alone function named build_expression_tree, which produces and returns an appropriate ExpressionTree instance, assuming the input has been tokenized.

```
1  def build_expression_tree(tokens):
2    """Returns an ExpressionTree based upon by a tokenized expression."""
3    S = [ ]                                    # we use Python list as stack
4    for t in tokens:
5      if t in '+-x*/':                         # t is an operator symbol
6        S.append(t)                            # push the operator symbol
7      elif t not in '()':                      # consider t to be a literal
8        S.append(ExpressionTree(t))            # push trivial tree storing value
9      elif t == ')':          # compose a new tree from three constituent parts
10       right = S.pop( )                       # right subtree as per LIFO
11       op = S.pop( )                          # operator symbol
12       left = S.pop( )                        # left subtree
13       S.append(ExpressionTree(op, left, right)) # repush tree
14     # we ignore a left parenthesis
15   return S.pop( )
```

Code Fragment 8.38: Implementation of a build_expression_tree that produces an ExpressionTree from a sequence of tokens representing an arithmetic expression.